

DeepPersona 코드 학습교재

모듈 03 — 시드/앵커 코어: based_data.py (Step 1)

Jin Sam Cho · cho@kaist.ac.kr

September 13, 2026

원저: Wang et al., *DeepPersona* (arXiv:2511.07338) · 논문 §3.2

이 모듈의 목표

한눈에 트리를 걷기 전에 필요한 **시드 인물(앵커)**을 만든다 — 이 시드가 이후 모든 선택·생성의 뿌리.

얻는 것

- 왜 코어를 먼저 고정하나(§3.2)
- **bias-free**(테이블) vs **LLM**(생성)의 구분
- 시드 의존성 체인(인구통계→이야기→취미)
- 각 함수 **코드블록 + 줄별 설명**
- ▶ 이 시드가 모듈 04의 임베딩 입력, 모듈 05의 값 생성 맥락이 된다.

핵심 통찰

- 나이·성별·위치·직업 = **LLM 안 씀**(편향 방지)
- 가치관 유형을 **랜덤(금/부/중)**으로 강제 → 낙관 편향 차단
- 취미는 이야기에서 **추론**(추출 아님)

1. 왜 코어를 먼저 고정하나
2. bias-free vs LLM
3. bias-free 코드
4. LLM 코드 — 성격·서사
5. 시드의 결과물

1. 왜 코어를 먼저 고정하나

안정된 코어를 앵커로 박는다

쉽게 말하면 집을 지을 때 기둥부터 세우듯, 페르소나도 흔들리지 않는 코어(나이·직업·가치관·이야기)를 먼저 정한다.

논문 §3.2 "Anchor a stable core"

- 먼저 소수의 코어를 인스턴스화:
나이·위치·직업·가치관·인생태도·인생이야기·취미
- 이걸 고정하면 selector가 말이 안 되는 영역으로 방황하지 않음

코어가 없으면?

LLM이 매 슬롯을 독립적으로 채워 앞뒤가 안 맞는 인물(30세인데 은퇴, 어부인데 도시 전문직)이 나옴.

코어가 있으면

이후 200개 슬롯이 모두 이 코어에 조건부로 생성
→ 일관성.

한눈에 based_data.py 의 8개 함수가 바로 이 코어(=Step 1 시드)를 만든다.

2. bias-free vs LLM

두 종류의 값 — 누가 정하는가

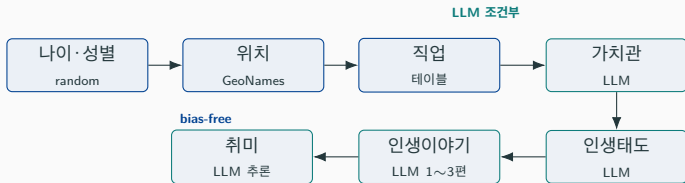
쉽게 말하면 인구통계는 "주사위(랜덤 테이블)", 성격·이야기는 "작가(LLM)". 섞지 않는 게 핵심.

항목	만드는 방법	왜 그렇게
나이·성별	random (범위/균등)	LLM은 특정 나이·성별에 쏠림
위치(국가·도시)	GeoNames DB 랜덤	실재 도시 → 현실성
직업	occupations.json 랜덤	다수문화 기본값 편중 방지
가치관·태도	LLM + 랜덤 유형	인구통계에 조건부 생성
인생이야기·취미	LLM	서사적 깊이는 LLM만 가능

한눈에 "Bias-free value assignment": 나이·직업·위치는 LLM이 아닌 테이블에서 뽑아 편향을 막는다.

시드 생성 의존성 체인 — 순서가 중요하다

쉽게 말하면 앞 단계의 결과가 다음 단계의 입력이 된다. 그래서 가치관에 맞는 이야기, 이야기에 맞는 취미가 나온다.



한눈에 인구통계 3개(파랑) → 성격·서사 4개(청록). 뒤로 갈수록 앞 정보를 맥락으로 받는다.

3. bias-free 코드

나이 · 직업 — LLM 없이 뽑는다

```
age = random.randint(7, 85)          # 균등랜덤 (LLM 아님)
if age <= 12: age_group = "child"
elif age <= 19: age_group = "adolescent"
elif age <= 29: age_group = "young_adult" # ... adult / middle_aged / senior

def generate_career_info(age):
    if age < 18 or age > 65:          # 학생은퇴/ -> 나이맞춰 LLM
        return {"status": get_completion(...)}
    occupations = get_occupations()   # occupations_english.json
    return {"status": random.choice(occupations)} # 그외 -> 테이블랜덤
```

- 나이: `random.randint(7,85)` — 7~85세 균등. LLM에 맡기면 특정 연령대로 쏠려서 **직접 랜덤**.
- `age_group`: 나이를 6개 구간으로 매핑(뒤 프롬프트에서 맥락으로 쓰임).
- 직업: 18~65세는 `occupations.json` 에서 `random.choice` — 다수문화 기본값 편중 방지.
- 예외: 미성년/고령만 LLM이 나이 맞는 "학생/은퇴" 상태 생성(테이블에 없는 케이스).

▶ `get_occupations()` 는 파일을 한 번 읽고 `_occupations_cache` 에 캐시(반복 I/O 방지).

위치 — 실재하는 도시를 뽑는다

```
gc = GeonamesCache()
countries = gc.get_countries()
country_code = random.choice(list(countries.keys())) # 국가랜덤
cities = gc.get_cities()
country_cities = [c for c in cities.values()
                  if c['countrycode'] == country_code] # 그나라의도시들
city_data = random.choice(country_cities) # 도시랜덤
return {"country": country['name'], "city": city_data['name']}
```

- **GeoNames**: 전 세계 국가·도시 오프라인 DB(`geonamescache` 패키지). 실재 지명만 나옴.
- **2단 랜덤**: 국가를 먼저 고르고 → 그 국가의 도시 중에서 고름(도시가 국가와 일치하도록).
- **왜 LLM 아님**: LLM은 유명 도시(뉴욕·런던)로 쏠림 → DB 랜덤이 **지리적 다양성** 보장.
- **인생이야기와 연결**: 뒤에서 "그 지역에서 실제로 일어날 법한" 이야기를 만드는 근거가 됨.

4. LLM 코드 — 성격·서사

가치관 — 긍정/부정/중립을 랜덤 주입

```
value_type = random.choice(['positive', 'negative', 'neutral']) # 핵심!
prompt = f"""... This person has a {value_type.upper()} value system ...
        format your response EXACTLY as JSON:
        {{ "values_orientation": "short phrase ..." }}"""
response = get_completion(messages, temperature=0.8)
result = parse_gpt_response(response, expected_fields=["values_orientation"], ...)
```

- `value_type` 랜덤: 긍정/부정/중립을 코드가 강제로 정해 프롬프트에 주입.
- 왜: 그냥 "가치관 만들어"라고 하면 LLM은 늘 긍정적·바람직한 값만 생성(낙관 편향). 부정도 섞어 다양성 확보.
- `temperature=0.8`: 다양한 표현이 나오도록 높게.
- 입력: 나이·성별·직업·위치(앞 단계 결과) → 인구통계에 조건부인 가치관.
- 출력: `{"values_orientation": "..."}` — 다음 단계(태도)의 입력.

▶ `parse_gpt_response` 는 마크다운/잡음 섞인 응답에서 JSON을 견고하게 추출(모듈 06에서 상술).

인생이야기 → 취미 — **추론**으로 잇는다

```
num_stories = random.randint(1, 3)          # 편1~3 무작위
prompt = f"""Generate {num_stories} personal stories ... (each 150-200 words)
... closely related to their country and region ...""" # 지역밀착
# --- 취미는이야기에서추론  '' ---
prompt = """... infer two to three hobbies ... rather than simply
extracting them directly from the story."""
```

- **이야기 1~3편**: 편수를 랜덤화 → 인물마다 서사 밀도가 다름.
- **지역 밀착**: "그 국가·지역에서 실제로 일어날 법한" 사건 요구 → 위치 앵커를 활용.
- **취미는 추론**: 이야기에서 직접 뽑지 말고 그 사람이 할 법한 취미를 **유추**(temperature=0.2 로 안정적).
- **왜 추론**: 이야기와 일관되면서도 명시되지 않은 취미까지 나와 3차원적 인물 완성.

한눈에 이렇게 논문의 "life-story-driven approach"로 코어에서 취미·관심까지 일관되게 확장한다.

5. 시드의 결과물

쉽게 말하면 위 함수들의 출력을 모으면 한 사람의 "씨앗 정보"가 된다. 이게 모듈 04에서 임베딩되어 슬롯 선택의 기준이 된다.

시드 필드(대략)

- `age` , `age_group` , `gender`
- `location` (country, city)
- `career` (status)
- `values_orientation`
- `attitude` , `attitude_details` , `coping_mechanism`
- `personal_story` (1~3편 결합)
- `interests` (2~3개)

다음 단계로

이 시드를 텍스트로 요약 →

`text-embedding-ada-002` 로 벡터화 → 2,297개
슬롯과 코사인 유사도(모듈 04).

논문 대응: 이 코어가 식 (2)의 앵커 S 그 자체.

한눈에 시드 = 식 (2)의 S . 이후 `selector · generator`가 전부 이 S 에 조건부로 동작.

꼭 기억할 것

- 코어를 **먼저 고정**(방향 방지)
- 인구통계 = **테이블/랜덤**(편향 차단)
- 성격·서사 = **LLM 조건부**
- 가치관 유형 **랜덤 주입**(낙관 편향↓)
- 취미는 이야기에서 **추론**

함수 지도(`based_data.py`)

- `generate_age_info` · `generate_gender`
- `generate_location` · `generate_career_info`
- `generate_personal_values`
- `generate_life_attitude`
- `generate_personal_story`
- `generate_interests_and_hobbies`

한눈에 다음 `04_select_attributes` : 이 시드로 2,297개 슬롯 중 200개를 **5:3:2**로 고른다.

모듈 03 끝

다음: 04_select_attributes — 임베딩 + 5:3:2 선택 (Step 2–4)